Software Architecture and Design Specification

Project: Task Manager Application

Version: 1.0

Authors:

PES2UG24CS561	Tharunyah Sudhakar
PES2UG24CS594	Vishnu Narayanan Ajay
PES2UG24CS597	Vismay Chandra Dev
PES2UG24CS606	Yajat Mathur

Date: 14-09-2026

Status: Completed

Revision History

Version	Date	Author	Change Summary
1.0	14-09-2026	Student Team	SAD completed for the Task Manager Application.

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification defines the architecture and design of the Task Manager Application. It translates the functional, non-functional, security, and interface requirements into architectural components, module responsibilities, interfaces, design flows, and traceability.

1.2 Scope

The design covers user registration and login; task creation, viewing, editing, deletion and completion; priority, status, category/tag and due-date management; reminders and notifications; search, filtering, sorting and progress summaries; user preferences; administrator functions where enabled; and protection of user/task data.

1.3 Audience

Developers, QA Engineers, UI/UX Designers, System Administrators, Project Managers, End Users, and Assessment Evaluators.

1.4 Definitions

Task – a unit of work to be completed; User – an authenticated application user; Priority – importance level assigned to a task; Status – current state of a task; Due Date – date/time by which a task should be completed; Reminder – notification associated with a task; UI – User Interface; API – Application Programming Interface; CRUD – Create, Read, Update and Delete; TLS – Transport Layer Security; RTM – Requirements Traceability Matrix.

2. Document Overview

2.1 How to use this document

This document describes the proposed architecture and design of the Task Manager Application. It provides the component/class view, sequence flows, interface contracts, security architecture, error handling, UX considerations, risks, and requirement traceability. Items not explicitly specified in the requirements are identified as proposed design decisions rather than existing implementation facts.

2.2 Related Documents

Primary source: Software Requirements Specification (SRS), Project: Task Manager Application, Version 1.0, dated 09-09-2026. The SRS defines the requirements, acceptance criteria, use-case diagrams, and RTM that this SAD maps to.

3. Architecture

3.1 Goals & Constraints

Goals:

- Provide a simple, responsive interface for managing personal tasks.
- Maintain data consistency for task creation, update, completion and deletion.
- Protect credentials and user/task data from unauthorized access or disclosure.

• Support reliable server-backed task management and graceful handling of temporary connectivity failures.

• Meet the performance, reliability, security, usability and accessibility acceptance criteria.

Constraints:

• Network connectivity is required for server-backed functions.

• The application must operate within supported browser/mobile/desktop, backend, database and network environments.

• No specialized hardware is required.

• Administrator capabilities are available only where administrator access is enabled.

• No concrete programming language, framework, database product, or cloud provider; those choices remain implementation decisions.

3.2 Stakeholders & Concerns

Stakeholder	Primary Concerns
User	Simple/responsive task management, search/filtering, reminders, privacy and reliable updates.
Administrator	Authorized user/account management, application monitoring and configuration where enabled.
Developer	Modularity, maintainability, clear interfaces and separation of responsibilities.
QA Engineer	Testability and traceability from requirements to design and test cases.
UI/UX Designer	Responsive UI, understandable validation/error messages and accessibility.
System Administrator	Availability, secure deployment, monitoring and recovery from temporary failures.

Assessment Evaluator	Completeness of architecture/design deliverables and traceability to requirements.

3.3 Component (UML) Diagram

The following logical component view is derived from the application's functions and stored data. It shows proposed logical modules and relationships; it does not claim a specific implementation technology. A supporting UML class diagram (Figure 2) then details the core domain entities, their attributes, operations, and relationships.

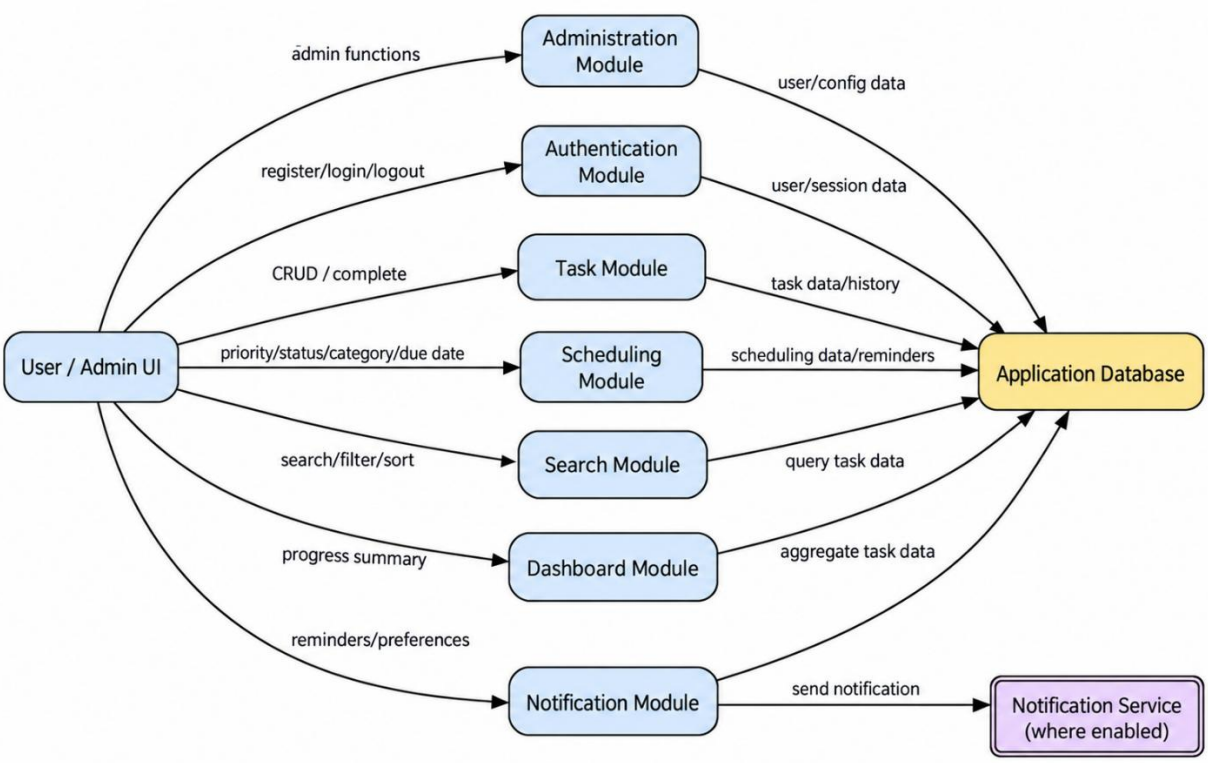


Figure 1. Proposed logical component architecture for the Task Manager Application.

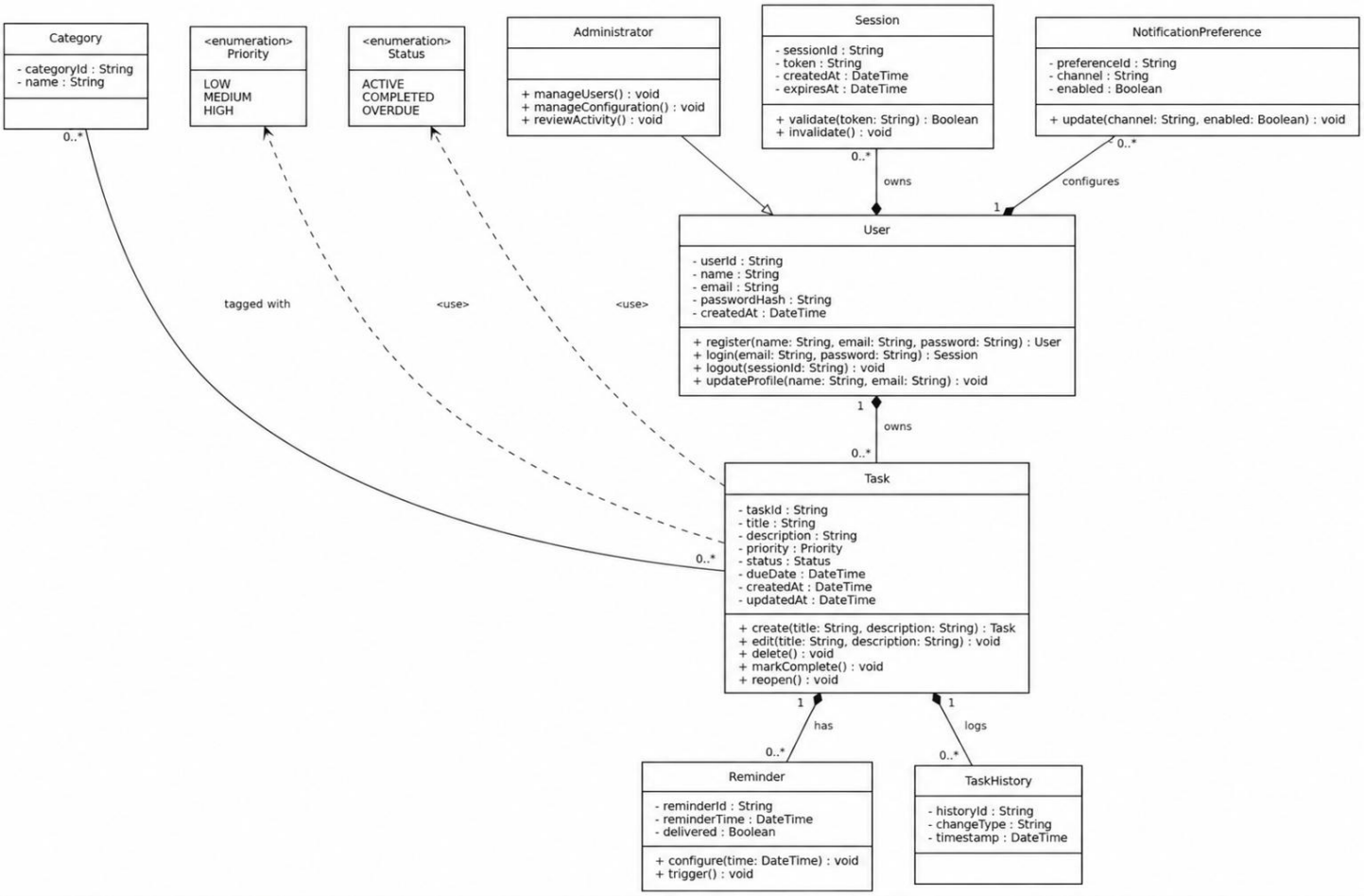


Figure 2. UML class diagram for the Task Manager Application, showing core entities, attributes, operations, relationships, and the Priority/Status enumerations.

3.4 Component Descriptions

Component	Responsibility
User / Admin UI	Provides dashboard, task lists, task-entry/detail views, filters, search, status/priority indicators, due dates and notification settings. Provides validation and understandable error messages.
Authentication Module	Handles registration, login, logout, password protection, session/token lifecycle and authentication-related security events.

Task Module	Provides task creation, viewing, editing, deletion, completion/reopening and ownership checks.
Scheduling Module	Manages priority, status, categories/tags, due dates and task reminders.
Search Module	Searches by title/description and filters/sorts by status, priority, category/tag and due date.
Dashboard Module	Produces summaries of active, completed and overdue tasks.
Notification Module	Stores reminder configuration and user notification preferences and triggers notifications where notification services are available.
Administration Module	Provides authorized administrator user/account management and application-level functions where enabled.
Application Database	Persists user accounts, tasks, categories, priorities, due dates, reminders, preferences and task history.

3.5 Chosen Architecture Pattern and Rationale

Proposed pattern: Layered architecture with modular services/components.

This choice follows the requirement for a UI plus backend data service and separates presentation, authentication/session management, task/business logic, search/notification/administration functions, and persistence.

It supports maintainability, testability and access-control boundaries. A microservices decomposition is not required and would add deployment and operational complexity for the stated scope, so the design keeps the logical modules within one application/backend unless future scale requires otherwise.

3.6 Technology Stack & Data Stores

No specific technology stack is prescribed. The architecture therefore specifies technology-neutral interfaces and a persistent application database. The implementation should provide:

• Client: supported web browser and/or mobile/desktop UI.

• Backend: application server exposing secure client-server interfaces.

• Database: persistent store for users, tasks, task metadata, reminders, preferences and task history.

• Communication: HTTPS/TLS for network communication.

• Authentication: secure password hashing and protected session credentials/tokens.

• Notification integration: notification service where enabled.

Concrete language/framework/database selections should be recorded as an implementation decision once the project stack is finalized.

3.7 Risks & Mitigations

Risk	Impact	Mitigation
Temporary network unavailability	Requests may fail or users may see stale state.	Handle failures gracefully; do not corrupt stored task data; retry safely where appropriate.
Unauthorized cross-user task access	Privacy and integrity violation.	Enforce authorization on every protected task operation and validate ownership server-side.
Credential compromise	Unauthorized account access.	Use HTTPS/TLS, secure password hashes, protected session credentials/tokens and invalidate sessions on logout/expiry.
Data inconsistency during task updates	Incorrect task state/history.	Use transactional persistence and verify database state after create/update/delete operations.
Notification delivery failure	Reminder may not reach the user.	Persist reminder configuration and expose notification status/errors; delivery depends on the enabled service.

Slow dashboard/search under load	Poor usability and failure of performance target.	Index/query task data appropriately, minimize unnecessary data transfer and measure response-time percentiles.

3.8 Traceability to Requirements

The detailed requirement-to-design mapping is provided in Section 5.4. Key examples include authentication requirements mapped to the Authentication Module, task CRUD/completion requirements mapped to the Task Module, and search/filter/sort requirements mapped to the Search Module.

3.9 Security Architecture

Security controls are based directly on the security objectives and requirements:

• Transport security: use HTTPS/TLS between supported clients and backend services (TMA-SR-001).

• Credential protection: store passwords only as secure password hashes; never store plaintext passwords (TMA-SR-002 and TMA-NF-003).

• Authorization: users may access and modify only permitted resources, including their own tasks (TMA-SR-003 / TMA-F-014).

• Session security: protect session credentials/tokens and invalidate them on logout or expiry (TMA-SR-004).

• Security logging: record failed logins and administrative actions without recording passwords (TMA-SR-005).

• Data integrity: preserve confirmed task changes during normal operation (TMA-NF-004).

STRIDE-oriented design review (proposed): Spoofing → authentication/session controls; Tampering → authorization, server-side validation and transactional updates; Repudiation → security-relevant audit logging; Information Disclosure → TLS and access control; Denial of Service → request validation and rate limiting where appropriate; Elevation of Privilege → role/authorization checks. Rate limiting is an additional design safeguard, not an explicit requirement.

4. Design

4.1 Design Overview

The application is designed as a layered set of logical modules. The UI communicates with the Authentication Module, Task Module, Scheduling Module, Search Module, Dashboard Module, Notification Module and Administration Module. These modules access the persistent database through controlled interfaces. Security and authorization checks are applied at the backend boundary so that client-side controls are not the only protection.

4.2 UML Sequence Diagrams

Two representative flows are included: (1) user login and task dashboard access, and (2) task creation. These flows are specific to the application's requirements.

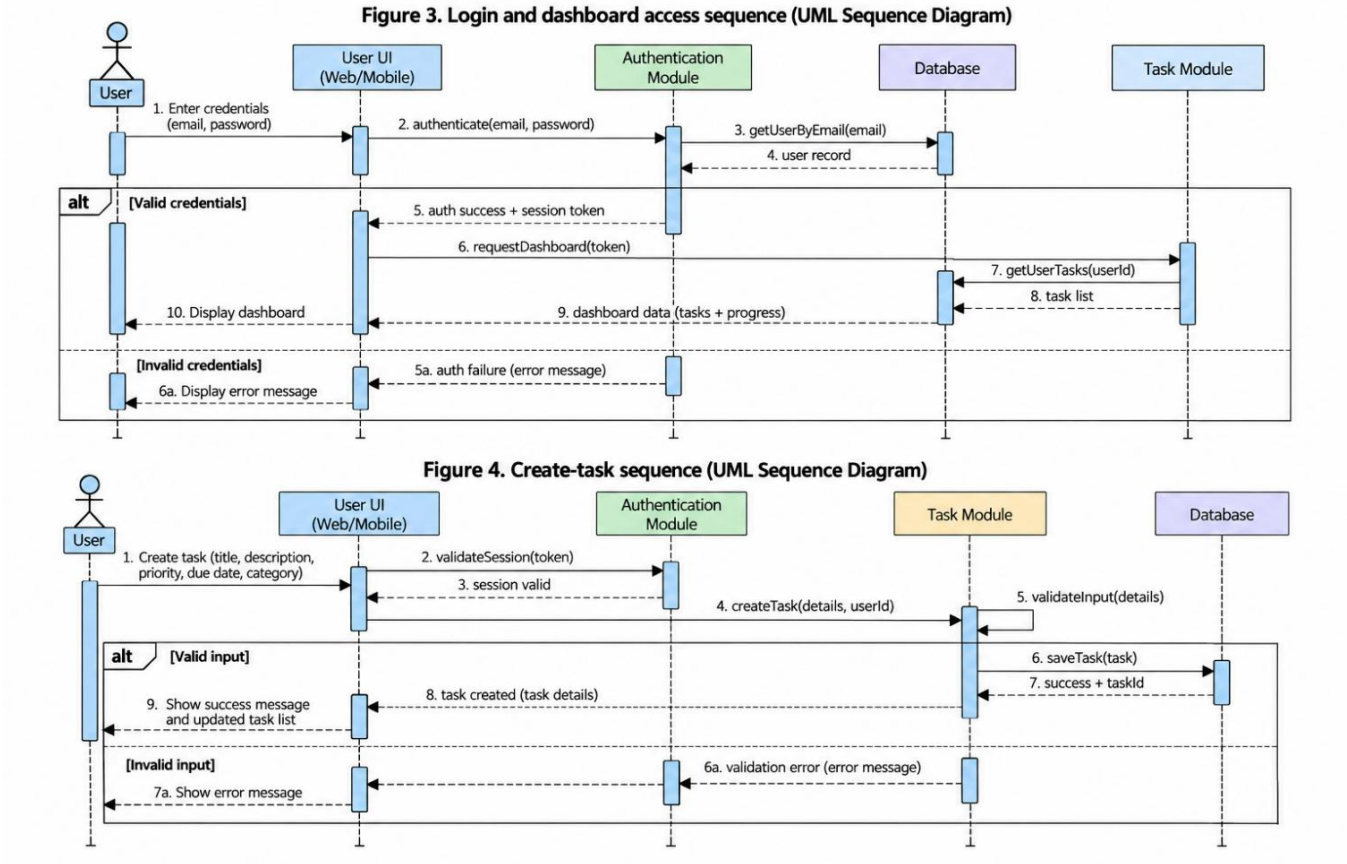


Figure 3. Login and dashboard access sequence

Figure 4. Create-task sequence

4.3 API Design

Concrete endpoints are not predefined. The following are proposed REST-style interface contracts for two logical components and are intended to satisfy the design interface requirement. Exact paths and payload fields may be adjusted during implementation.

Component / Endpoint	Method	Request	Response	Errors / Security
Authentication Module /api/auth/login	POST	{email, password}	{status, userId, sessionToken}	401 Invalid credentials; do not expose password details. Session token protected.
Task Module /api/tasks	POST	{title, description?, priority?, status?, category?, dueDate?}	{status, taskId, task}	400 Validation error; 401/403 unauthenticated or unauthorized. Authenticated user context required.
Task Module /api/tasks/{taskId}	PATCH	Fields to update	{status, task}	400 Validation; 401/403 unauthorized; 404 task not found.
Task Module /api/tasks/{taskId}	DELETE	No body	{status}	401/403 unauthorized; 404 task not found; confirmation required at UI level.

Additional logical interfaces include search/filter/sort queries, reminder configuration, notification preferences, progress-summary retrieval, logout/session invalidation, and administrator user/account management.

4.4 Error Handling, Logging & Monitoring

• Validate required fields and return clear, understandable error messages.

• Use consistent API error structures across modules.

• Reject unauthorized cross-user resource requests rather than returning another user's data.

• Do not log passwords, session credentials/tokens, or other sensitive credentials.

• Record security-relevant events such as failed logins and administrative actions.

• Monitor dashboard response time, application availability, task-operation failures, notification failures and relevant security events.

• Handle temporary network failures without corrupting stored task data.

4.5 UX Design

The UI should provide dashboards, task lists, task-entry forms, task-detail views, filters, search controls, status and priority indicators, due dates and notification settings. It should be responsive on supported desktop and mobile screen sizes and provide accessible labels and controls where applicable. Validation and error messages should explain what the user needs to correct. These choices directly reflect the interface and usability/accessibility requirements.

4.6 Open Issues & Next Steps

• Finalize the implementation technology stack, database product and deployment environment; these are not yet finalized.

• Finalize the exact API paths, payload schemas and authentication mechanism during implementation.

• Decide whether synchronization across sessions/devices is required in the implemented scope.

• Select and configure the notification service where reminders are enabled.

• Implement and execute the test cases listed in the RTM and update status from N (Not started) as verification progresses.

• Review the architecture against performance, reliability, security and accessibility acceptance criteria before final acceptance.

5. Appendices

5.1 Glossary

Term	Meaning
API	Application Programming Interface.
CRUD	Create, Read, Update and Delete.
RTM	Requirements Traceability Matrix.
Task	A unit of work to be completed.
TLS	Transport Layer Security used to protect network communication.
UI	User Interface.
User	An authenticated application user.
Reminder	A notification associated with a task.

5.2 References

1. UML use-case diagrams and Requirements Traceability Matrix referenced in Section 2.2 (Related Documents).

2. SRS Dated 09-09-2026

5.3 Tools

Document authoring: Microsoft Word

Diagrams: PlantUML and draw.io.

API design: Swagger

5.4 Requirements Traceability Matrix (Reference for Section 3.8)

Req ID	Requirement Short	Section Ref / Design Spec	Module	Test Case(s)
TMA-F-001	User registration	4.1 / DS-Auth-01	Authentication Module	TC-Auth-01
TMA-F-002	User login	4.1 / DS-Auth-02	Authentication Module	TC-Auth-02

TMA-F-003	Invalid login rejection	4.1 / DS-Auth-03	Authentication Module	TC-Auth-03
TMA-F-004	User logout	4.1 / DS-Auth-04	Authentication Module	TC-Auth-04
TMA-F-010	Create task	4.2 / DS-Task-01	Task Module	TC-Task-01
TMA-F-011	Edit task	4.2 / DS-Task-02	Task Module	TC-Task-02
TMA-F-012	Delete task	4.2 / DS-Task-03	Task Module	TC-Task-03
TMA-F-013	Mark task complete/reopen	4.2 / DS-Task-04	Task Module	TC-Task-04
TMA-F-014	Task access authorization	4.2 / DS-Sec-01	Access Control	TC-Sec-01
TMA-F-020	Task priority	4.3 / DS-Org-01	Task Module	TC-Org-01
TMA-F-021	Task status update	4.3 / DS-Org-02	Task Module	TC-Org-02
TMA-F-022	Task categories/tags	4.3 / DS-Org-03	Task Module	TC-Org-03
TMA-F-023	Due date	4.3 / DS-Org-04	Scheduling Module	TC-Org-04
TMA-F-024	Task reminder	4.3 / DS-Org-05	Notification Module	TC-Org-05
TMA-F-030	Task search	4.4 / DS-Search-01	Search Module	TC-Search-01
TMA-F-031	Task filtering	4.4 / DS-Search-02	Search Module	TC-Search-02
TMA-F-032	Task sorting	4.4 / DS-Search-03	Search Module	TC-Search-03
TMA-F-033	Progress summary	4.4 / DS-Progress-01	Dashboard Module	TC-Progress-01

TMA-F-040	Notifications	4.5 / DS-Notify-01	Notification Module	TC-Notify-01
TMA-F-041	Notification preferences	4.5 / DS-Notify-02	Notification Module	TC-Notify-02
TMA-F-042	Admin user management	4.5 / DS-Admin-01	Administration Module	TC-Admin-01
TMA-NF-001	Dashboard response time	5 / DS-Perf-01	Application/UI	TC-Perf-01
TMA-NF-002	System availability	5 / DS-Reliability-01	Infrastructure/Ops	TC-Reliability-01
TMA-NF-003	Data security	5 / DS-Sec-01	Security Module	TC-Sec-02
TMA-NF-004	Data consistency	5 / DS-Integrity-01	Database/Task Module	TC-Integrity-01
TMA-NF-005	UI responsiveness/accessibility	5 / DS-UX-01	UI Module	TC-UX-01
TMA-SR-001	HTTPS/TLS	5.1.2 / DS-NetSec-01	Network Security	TC-Sec-03
TMA-SR-002	Password hashing	5.1.2 / DS-Sec-02	Authentication Module	TC-Sec-02
TMA-SR-003	Authorization	5.1.2 / DS-Access-01	Access Control	TC-Sec-01
TMA-SR-004	Session token protection	5.1.2 / DS-Sec-03	Session Management	TC-Sec-04
TMA-SR-005	Security event logging	5.1.2 / DS-Sec-04	Security/Audit	TC-Sec-05